Practical No 05

A) Create a web application to demonstrate the use of different types of Cookies.

Project Name: Cookies

Project Type: ASP.NET Web Application (.NET Framework)

Template: Web Forms

Program Code:

I.Filename: CookiesDemo.aspx.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;

namespace Cookies
{
    public partial class CookiesDemo : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }

        protected void btnSave_Click(object sender, EventArgs e)
        {
            HttpCookie sessionCookie = new HttpCookie("SessionUser");
            sessionCookie.Value = txtName.Text;
            Response.Cookies.Add(sessionCookie);

            HttpCookie persistentCookie = new HttpCookie("PersistentUser");
            persistentCookie.Value = txtName.Text;
            persistentCookie.Expires = DateTime.Now.AddDays(7);
            Response.Cookies.Add(persistentCookie);

            lblOutput.Text = "cookies Saved Successfully";
        }

        protected void btnShow_Click(object sender, EventArgs e)
        {
            string sessionData = Request.Cookies["SessionUser"].Value;
            string persistentData = Request.Cookies["PersistentUser"].Value;

            lblOutput.Text = "Session Cookie: " + sessionData +
                             "<br/>Persistent Cookie: " + persistentData;
        }
    }
}
```

II.Filename: CookiesDemo.aspx

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeBehind="CookiesDemo.aspx.cs"
    Inherits="Cookies.CookiesDemo" %>
```

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div>
            <asp:Label
                ID="Label1"
                runat="server"
                Text="User Name:">
            </asp:Label>
             
            <asp:TextBox
                ID="txtName"
                runat="server">
            </asp:TextBox>
            <br />
            <br />
            <asp:Button
                ID="btnSave"
                runat="server"
                OnClick="btnSave_Click"
                Text="Save cookie" />
            <br />
            <br />
            <asp:Button
                ID="btnShow"

                runat="server"
                OnClick="btnShow_Click"
                Text="Show Cookie" />
            <br />
            <br />
            <asp:Label
                ID="lblOutput"
                runat="server"
                Text="Output">
            </asp:Label>
            <br />
        </div>
    </form>
</body>
</html>
```

B) Create a Product Table in SQL Server and perform Insert Operation using Entity Framework Core.

Project Name: ProductEFCore

Project Type: ASP.NET Core Web App (MVC**)**

Program Code:

I.Filename: Products.cs

```
namespace ProductEFCore.Models
{
    public class Products
    {
        public int Id { get; set; }
        public required string ProductName { get; set; }
        public double Price { get; set; }
    }
}
```

II.Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;

namespace ProductEFCore.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)

            : base(options)
        {
        }
        public DbSet<Products> Products { get; set; }
    }
}
```

II.Filename: Program.cs

```
using ProductEFCore.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

III.Filename: appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },

  "AllowedHosts": "*",
```

```
  "ConnectionStrings": {
    "MyConnection": "Server=(Localdb)\\MSSQLLocalDB; Database=ProductDB; Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

IV.Filename: ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ProductEFCore.Models;
using ProductEFCore.Data;

public class ProductsController : Controller
{
    private readonly AppDbContext _context;

    public ProductsController(AppDbContext context)
    {
        _context = context;
    }

    public async Task<IActionResult> Index()
    {
        return View(await _context.Products.ToListAsync());
    }

    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var products = await _context.Products
            .FirstOrDefaultAsync(m => m.Id == id);

        if (products == null)
        {
            return NotFound();
        }

        return View(products);
    }

    public IActionResult Create()
    {
```

```
        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(
        [Bind("Id,ProductName,Price")] Products products)
    {
        if (ModelState.IsValid)
        {
            _context.Add(products);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }

        return View(products);
    }

    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var products = await _context.Products.FindAsync(id);

        if (products == null)
        {
            return NotFound();
        }

        return View(products);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(
        int? id,
        [Bind("Id,ProductName,Price")] Products products)
    {
        if (id != products.Id)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
```

```
            try
            {
                _context.Update(products);
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!ProductsExists(products.Id))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }

            return RedirectToAction(nameof(Index));
        }

        return View(products);
    }

    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var products = await _context.Products
            .FirstOrDefaultAsync(m => m.Id == id);

        if (products == null)
        {
            return NotFound();
        }

        return View(products);
    }

    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int? id)
    {
        var products = await _context.Products.FindAsync(id);

        if (products != null)
```

```
        {
            _context.Products.Remove(products);
            await _context.SaveChangesAsync();
        }

        return RedirectToAction(nameof(Index));
    }

    private bool ProductsExists(int? id)
    {
        return _context.Products.Any(e => e.Id == id);
    }
}
```

C) Develop a web application to perform CRUD Operations (Create, Read, Update, Delete) on Employee table using EF Core.

Project Name: AuthDemo

Project Type: ASP.NET Core Web App (MVC**)**

Program Code:

I.Filename: Employees.cs

```
namespace EmployeeCRUD.Models
{
    public class Employees
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public double Salary { get; set; }
    }
}
```

II.Filename: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCURD.Models;

namespace EmployeeCURD.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }
        public DbSet<Employees> Employees { get; set; }
    }
}
```

III.Filename: Program.cs

```
using Microsoft.EntityFrameworkCore;
using EmployeeCURD.Data;

var builder = WebApplication.CreateBuilder(args);



builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(
        builder.Configuration.GetConnectionString("MyConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();
```

IV.Filename: appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
```

```
    "MyConnection": "Server=(Localdb)\\MSSQLLocalDB; Database=EmployeeDB; Trusted_Connection=True;
TrustServerCertificate=True"
  }
}
```

V.Filename: EmployeesController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using EmployeeCRUD.Models;
using EmployeeCURD.Data;

public class EmployeesController : Controller
{
    private readonly AppDbContext _context;

    public EmployeesController(AppDbContext context)
    {
        _context = context;
    }



    public async Task<IActionResult> Index()
    {
        return View(await _context.Employees.ToListAsync());
    }

    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var employees = await _context.Employees
            .FirstOrDefaultAsync(m => m.Id == id);

        if (employees == null)
        {
            return NotFound();
        }

        return View(employees);
    }



    public IActionResult Create()
```

```
    {
        return View();
    }


    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(
        [Bind("Id,Name,Department,Salary")] Employees employees)
    {
        if (ModelState.IsValid)
        {
            _context.Add(employees);
            await _context.SaveChangesAsync();

            return RedirectToAction(nameof(Index));
        }

        return View(employees);
    }

    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var employees = await _context.Employees.FindAsync(id);

        if (employees == null)
        {
            return NotFound();
        }

        return View(employees);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(
        int? id,
        [Bind("Id,Name,Department,Salary")] Employees employees)
    {
        if (id != employees.Id)
        {
            return NotFound();
        }
```

```
        if (ModelState.IsValid)
        {
            try
            {
                _context.Update(employees);
                await _context.SaveChangesAsync();
            }
            catch (DbUpdateConcurrencyException)
            {
                if (!EmployeesExists(employees.Id))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }
            return RedirectToAction(nameof(Index));
        }
        return View(employees);
    }


    public async Task<IActionResult> Delete(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }
        var employees = await _context.Employees
            .FirstOrDefaultAsync(m => m.Id == id);
        if (employees == null)
        {
            return NotFound();
        }
        return View(employees);
    }

    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int? id)
    {
        var employees = await _context.Employees.FindAsync(id);
        if (employees != null)
        {
            _context.Employees.Remove(employees);
        }
```

```
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    private bool EmployeesExists(int? id)
    {
        return _context.Employees.Any(e => e.Id == id);
    }
}
```

D) Authentication and Authorization in ASP.NET Core

Project Name: AuthDemo

Project Type: ASP.NET Core Web App (MVC**)**

Program Code:

I.Filename: Secret.cshtml

```
@{
    ViewData["Title"] = "Secret";
}
<h2>Authorized Users Only</h2>
```

II.Filename: HomeController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace AuthDemo.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [Authorize]
        public IActionResult Secret()
        {
            return View();
        }
    }
}
```

III.Filename: Program.cs

```
using AuthDemo.Data;
using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.EntityFrameworkCore;



var builder = WebApplication.CreateBuilder(args);
// Add services to the container.
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException(
        "Connection string 'DefaultConnection' not found.");
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));
builder.Services.AddDatabaseDeveloperPageExceptionFilter();
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
    options.SignIn.RequireConfirmedAccount = true)
    .AddEntityFrameworkStores<ApplicationDbContext>();
builder.Services.AddControllersWithViews();
var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseMigrationsEndPoint();
}
else
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapRazorPages();
app.Run();
```